DATA 260

HOMEWORK 9

Redis Semantic Caching and Self-Improving Multi-Agent System

Shoury Ambarish Parab

SID4: 7085

Repository: https://github.com/shoddy16/data260-7085

Setting	Verified value
PREFIX	s7085
SEED	7085
VERIFY_SEED	267085
DOMAIN_ID	5
PORT_BASE / HW9_PORT	8085 / 8086
Chat / embedding	llama3.2:3b / nomic-embed-text
Embedding dimension	768
Backend	FastAPI, SQLAlchemy, Redis Stack, Ollama

This report presents my completed HW9 implementation and the evidence collected for each part. It distinguishes live Redis/Ollama/MySQL results, deterministic local experiments, unit tests, and reproducibility artifacts.

Contents

1. Project context and architecture
2. Part 1 - Redis restaurant view counter
3. Part 2 - Redis semantic response cache
4. Part 3 - Frozen dataset and threshold economics
5. Stale-answer invalidation
6. Part 4 - Self-improving agents
7. Testing and regression audit
8. Required artifacts and reproducibility
9. AI use and verification
10. Final checklist and conclusion

1. Project context and architecture

I extended the existing restaurant-inspection application rather than creating a separate project. HW9 adds a Redis view counter, a semantic response cache in front of the existing Q&A path, a frozen threshold experiment, and a reference-backed evaluator loop. The previous HW2 report used PORT_BASE=8785. The current HW9 configuration is verified in hw9_config.py and Compose: PORT_BASE=8085 and HW9_PORT=8086, with Redis exposed on host port 6380.

The request flow is: restaurant access -> Redis INCR -> periodic database synchronization. A question -> Ollama embedding -> Redis HNSW KNN -> similarity and safeguards -> cached answer or existing answer path. The self-improvement experiment is separate from live model latency and uses reference data to make its outcomes repeatable.

```
001: """HW9 configuration derived from the assigned HW8 project."""
002:
003: SID4 = 7085
004: PORT_BASE = 8085
005: HW9_PORT = 8086
006: PREFIX = "s7085"
007: SEED = 7085
008: VERIFY_SEED = 267085
009: DOMAIN_ID = 5
010: DOMAIN = "local restaurant inspections"
011: LOCAL_MODEL = "llama3.2:3b"
012: EMBEDDING_MODEL = "nomic-embed-text"
013: EMBEDDING_DIM = 768
014: OLLAMA_URL = "http://localhost:11434"
015: REDIS_URL = "redis://localhost:6379/0"
016: VIEW_KEY = f"{PREFIX}:views:restaurant"
017: CACHE_KEY = f"{PREFIX}:semantic_cache"
018: DATA_VERSION_KEY = f"{PREFIX}:data_version:restaurant"
019: DEFAULT_THRESHOLD = 0.85
```

Figure 1. Configuration excerpt from hw9_config.py.

The hardware query was restricted in this environment, so I do not report an unverified CPU or memory specification. Python 3.10.2 was observed during the earlier live validation. Docker Engine 29.9.2 and the local Ollama runtime were available for the live Redis and embedding checks.

2. Part 1 - Redis-backed restaurant view counter

Requirement. Every restaurant access must increment Redis with INCR, use the s7085 prefix, synchronize the relational row every tenth view, return the current count, and handle missing records, Redis failures, rollback, and concurrency.

Implementation. ViewCounter.increment_restaurant_views builds s7085:views:restaurant:{id}. Redis INCR is atomic for one key, so concurrent requests receive distinct counts. The database is touched only when count % 10 == 0. This reduces database write pressure while keeping the twentieth view persisted.

```
042:     def increment_restaurant_views(self, restaurant_id: int, db: Any) -> int:
043:         key = f"{VIEW_KEY}:{restaurant_id}"
044:         try:
045:             count = int(self.redis.incr(key))
046:         except Exception as exc:
047:             log.exception("Redis view increment failed for restaurant %s", restaurant_id)
048:             raise RuntimeError("Unable to increment restaurant views") from exc
049:         if count % 10 == 0:
050:             restaurant = db.query(RestaurantModel).filter(RestaurantModel.id ==
    restaurant_id).first()
051:             if restaurant is None:
052:                 raise ValueError(f"Restaurant {restaurant_id} not found")
053:             restaurant.views = count
054:             try:
055:                 db.commit()
056:                 log.info("Synced views to database restaurant_id=%s count=%s", restaurant_id,
    count)
057:             except Exception:
058:                 db.rollback()
059:                 log.exception("View database sync failed restaurant_id=%s count=%s",
    restaurant_id, count)
060:                 raise
061:         return count
```

The code catches Redis errors and raises a runtime error for the route to translate into HTTP 503. A missing Restaurant row raises ValueError. A failed commit rolls back the session before re-raising.


```
{
  "redis": "PASS",
  "mysql": "PASS",
  "prefix": "s7085",
  "timestamp": "2026-11-05T23:22:46.572718+00:00",
  "restaurant_id": 1,
  "counts_1_2_10_20_21": [
    1,
    2,
    10,
    20,
    21
  ],
  "database_views_after_21": 20,
  "redis_key": "s7085:views:restaurant:1",
  "tenth_sync": true,
  "concurrent_restaurant_id": 2,
  "concurrent_redis_count": 20,
  "concurrent_database_views": 20,
  "concurrent_unique_counts": 20
}
```


Figure 2. Live Redis/MySQL view-counter output.

The recorded counts were 1, 2, 10, 20, and 21. The database remained at 20 after the twenty-first access, which is the expected every-tenth-write behavior. The concurrent run produced 20 unique Redis counts and database_views=20.

2.1 API route, errors, and concurrency

```
114: def increment_views(
115:     restaurant_id: int,
116:     db: Session = Depends(get_db),
117:     _user=Depends(get_current_user),
118: ):
119:     try:
120:         views = ViewCounter(redis_client).increment_restaurant_views(restaurant_id, db)
121:     except ValueError as exc:
122:         raise HTTPException(status_code=404, detail=str(exc)) from exc
123:     except RuntimeError as exc:
124:         raise HTTPException(status_code=503, detail=str(exc)) from exc
125:     return ViewResponse(restaurant_id=restaurant_id, views=views)
126:
127:
128: @router.get("/health")
```

The route injects a database session and the current user, then maps ValueError to 404 and Redis/runtime failures to 503. The response returns restaurant_id and views. The focused tests also exercise independent IDs and a broken Redis client. The live evidence and the in-memory concurrency test support the atomic-increment claim; they do not claim that the database is synchronized on every request.

Check	Observed result
Redis key	s7085:views:restaurant:1
Counts	[1, 2, 10, 20, 21]
Database after 21	20
Tenth sync	True
Concurrent unique counts	20
Concurrent database views	20

Figure 3. Part 1 verification table.

3. Part 2 - Redis semantic response cache

The cache sits at the ingress of /hw9/ask. It embeds the incoming question with nomic-embed-text, searches the Redis HNSW index, converts Redis cosine distance to similarity, checks the threshold, and then applies query-signature, restaurant-scope, and data-version safeguards. A hit returns the stored answer without calling the answerer. A miss invokes the existing agent path and stores the answer and metadata.

The field alias embedding_vector is important. The previous index used the JSON path as its identifier and returned no usable KNN matches. I migrated the index without deleting JSON records and recreated it with an explicit alias and DIM=768.

```
141:         self.provider = provider or EmbeddingProvider()
142:         self.index_name = index_name or f"{PREFIX}:semantic_cache_idx_v{EMBEDDING_DIM}"
143:         self.index_prefix = index_prefix
144:         self.vector_field = "embedding_vector"
145:         self.knn_available = False
146:         self.last_lookup_backend = "unavailable"
147:         self._ensure_index()
148:
149:     def _ensure_index(self) -> None:
150:         if not hasattr(self.redis, "ft") or self.redis.__class__.__name__ == "MemoryRedis":
151:             self.last_lookup_backend = "offline_test_backend"
152:             return
153:         try:
154:             from redis.commands.search.field import TagField, TextField, VectorField
155:             from redis.commands.search.index_definition import IndexDefinition, IndexType
156:
157:             fields = [TextField("$.query"), TagField("$.signature"),
        TextField("$.data_version"),
158:                       VectorField("$.embedding", "HNSW", {"TYPE": "FLOAT32", "DIM":
        EMBEDDING_DIM,
159:                                                           "DISTANCE_METRIC": "COSINE"},
160:                                   as_name=self.vector_field)]
161:             index = self.redis.ft(self.index_name)
162:             try:
163:                 info = index.info()
164:                 vector_attrs = []
165:                 for attribute in info.get("attributes", []):
166:                     values = dict(zip(attribute[::2], attribute[1::2]))
167:                     if values.get(b"type") == b"VECTOR":
168:                         vector_attrs.append(values)
169:                 vector_info = vector_attrs[0] if vector_attrs else {}
170:                 identifier = vector_info.get(b"identifier", b"").decode()
171:                 dimension = int(vector_info.get(b"dim", 0))
172:                 if identifier != self.vector_field or dimension != EMBEDDING_DIM:
173:                     index.dropindex(delete_documents=False)
174:                     index.create_index(fields, definition=IndexDefinition(
175:                         prefix=[f"{self.index_prefix}:"], index_type=IndexType.JSON))
176:             except Exception as info_exc:
177:                 message = str(info_exc).lower()
178:                 if "unknown index name" not in message and "no such index" not in message:
```

3.1 KNN lookup and similarity conversion

```
238:             if (record.get("data_version") != current_version
239:                     or record.get("signature") != signature
240:                     or record.get("restaurant_id") != restaurant_id):
241:                 continue
242:             score = cosine_similarity(embedding, record.get("embedding", []))
243:             if score > best[1]:
244:                 best = (record | {"cache_id": cache_id}, score)
245:         return best if best[1] >= self.threshold else (None, best[1])
246:
247:     def _redis_knn_lookup(
248:         self, signature: str, embedding: list[float], restaurant_id: int | None,
    current_version: str
249:     ) -> tuple[dict[str, Any] | None, float]:
250:         from redis.commands.search.query import Query
251:
252:         query = Query(f"*=>[KNN 1000 @{self.vector_field} $query_vector AS vector_distance]")
253:         query = query.sort_by("vector_distance").return_fields("vector_distance").paging(0,
    1000).dialect(2)
254:         packed = struct.pack(f"{len(embedding)}f", *embedding)
255:         try:
256:             result = self.redis.ft(self.index_name).search(query, query_params={"query_vector":
    packed})
257:         except Exception as exc:
258:             log.error("Redis KNN query failed: %s", exc)
259:             raise RuntimeError("Redis KNN query failed") from exc
260:         best: tuple[dict[str, Any] | None, float] = (None, 0.0)
261:         for document in result.docs:
262:             record = self._get_record(document.id)
263:             if not record:
264:                 continue
265:             if (record.get("data_version") != current_version
266:                     or record.get("signature") != signature
267:                     or record.get("restaurant_id") != restaurant_id):
268:                 continue
269:             raw_distance = getattr(document, "vector_distance", None)
270:             if raw_distance is None:
271:                 continue
272:             try:
273:                 similarity = max(0.0, min(1.0, 1.0 - float(raw_distance)))
274:             except (TypeError, ValueError):
```

Redis Search returns vector_distance for a COSINE index. The implementation computes 1 - distance, clamps the value to the valid similarity range, and rejects malformed distances. It then loads the document by Redis ID and checks data_version, signature, and restaurant_id before accepting the result. A live Redis client never silently falls back to a Python scan. The only explicit scan fallback is the MemoryRedis unit-test backend, which is labeled offline_test_backend.

```
{
  "backend": "redis_knn",
  "index_name": "s7085:semantic_cache_idx_v768",
  "vector_attribute": {
    "identifier": "$.embedding",
    "attribute": "embedding_vector",
    "type": "VECTOR",
    "algorithm": "HNSW",
    "data_type": "FLOAT32",
    "dim": 768,
    "distance_metric": "COSINE",
    "M": 16,
    "ef_construction": 200
  },
  "embedding_dimension": 768,
  "cases": [
    {
      "label": "exact",
      "query": "Which restaurants scored below 70?",
      "signature": "knn_exact",
      "is_cached": true,
      "similarity": 1.0,
      "backend": "redis_knn",
      "matched_query": "Which restaurants scored below 70?"
    },
    {
      "label": "paraphrase",
      "query": "List restaurants with scores under 70",
      "signature": "knn_exact",
      "is_cached": true,
      "similarity": 0.9385986328125,
      "backend": "redis_knn",
      "matched_query": "Which restaurants scored below 70?"
    },
    {
      "label": "logical_difference",
      "query": "Which restaurants scored above 70?",
      "signature": "knn_no_match",
      "is_cached": false,
      "similarity": 0.0,
      "backend": "redis_knn",
      "matched_query": null
    },
    {
      "label": "unrelated",
      "query": "Explain binary search",
      "signature": "knn_no_match",
      "is_cached": false,
      "similarity": 0.0,
      "backend": "redis_knn",
      "matched_query": null
    }
  ],
  "stale_guard": {
    "before_cached": true,
    "before_similarity": 1.0,
    "new_version": "1",
    "after_cached": false,
    "after_similarity": 0.0
  },
  "concurrent": {
    "workers": 4,
    "queries": 8,
    "all_backend_knn": true,
    "all_cached": true,
    "similarities": [
      1.0,
      1.0,
      1.0,
      1.0,
      1.0,
      1.0,
      1.0,
      1.0
    ]
  },
  "empty_index": {
    "is_cached": false,
    "similarity": 0.0,
    "backend": "redis_knn"
  },
  "status": "PASS"
}
```


Figure 4. Live Redis KNN validation output, including index metadata and backend confirmation.

3.2 Cache integration and safeguards

```
096:
097:
098: @router.post("/ask", response_model=AskResponse)
099: def ask(payload: AskRequest):
100:     def run_agent_once() -> str:
101:         result = system.run(payload.question)
102:         return result.get("revised_answer") or result["initial_answer"]
103:
104:     result = semantic_cache.answer(
105:         payload.question,
106:         query_signature(payload.question),
107:         run_agent_once,
108:         payload.restaurant_id,
109:     )
110:     return AskResponse(**result.__dict__)
111:
```

The route derives a coarse query signature for score predicates. This does not replace semantic similarity; it prevents an above query from reusing a below query answer. The restaurant_id check prevents cross-restaurant reuse. The data-version value is captured on write and compared during lookup. These guards are necessary because embeddings measure language similarity, not whether two questions have identical answer semantics.

Live case	Similarity	Cached	Agent executed
exact seed	0.000000	False	True
exact duplicate	1.000000	True	False
paraphrase	0.938599	True	False
similar different answer	0.000000	False	True
unrelated	0.000000	False	True

Figure 5. Live semantic-cache cases.

3.3 Embedding, empty-index, and concurrent validation

The live KNN validation used the configured 768-dimensional embedding model. Exact and paraphrase queries matched the temporary restaurant-scoped record. Logical and unrelated queries were rejected by signature filtering. After incrementing the temporary restaurant version, the old record was rejected. An isolated empty index produced a miss, and eight concurrent queries all reported the Redis KNN backend.

Case	Result
Exact	hit, similarity 1.0
Paraphrase	hit, similarity 0.9385986
Logical difference	miss
Unrelated	miss
Stale version	rejected after version increment
Empty index	miss
Concurrent	8/8 KNN lookups

Redis errors are surfaced as RuntimeError rather than silently producing a scan result. Invalid embedding vectors are rejected before packing or ignored when Redis returns an invalid distance.

4. Part 3 - Frozen dataset and threshold economics

The qualifying experiment used the committed cache_cases.yaml at commit a7533e7d5eb1df286b77450ce7999d0ad6a5a12d. The file contains 40 restaurant-inspection cases: ten exact duplicates, ten paraphrases, ten similar questions with different answers, and ten unrelated questions. The expected distribution is 20 hits and 20 misses. No labels were changed after the preliminary sweep.

Category	Count	Expected
Exact duplicate	10	hit
Paraphrase	10	hit
Similar, different answer	10	miss
Unrelated	10	miss
Total	40	20 hit / 20 miss

Figure 6. Frozen case distribution.

The experiment stores one set of reference answers, runs the same 40 cases at each threshold, and uses the live Ollama provider and Redis KNN backend for every row. Hit rate is hits/40. False-hit rate is false hits/20 expected-miss cases. Cost is (40 - hits)*1 + false_hits*8.

4.1 Qualifying five-threshold sweep

Threshold	Hits	Misses	Correct	False	Missed hits	Hit rate	False/20	Mean ms	Cost
0.75	29	11	20	9	0	0.725	0.450	2537.92	83
0.80	28	12	20	8	0	0.700	0.400	103.61	76
0.85	27	13	19	8	1	0.675	0.400	151.57	77
0.90	22	18	16	6	4	0.550	0.300	184.15	66
0.95	13	27	10	3	10	0.325	0.150	240.44	51

Figure 7. Final qualifying threshold results.

The cost decreases from 83 to 51 as the threshold increases. The decrease is not free: 0.95 keeps only 13 hits and misses ten cases that were labeled expected hits. The lowest-cost threshold is therefore a measured choice under this cost model, not a universal correctness guarantee.

4.2 False-hit analysis and latency interpretation

The qualifying sweep recorded a real live false hit at every threshold. At 0.75, the cached question was 'Which restaurant scored above 90?' and the incoming question was 'Which restaurants scored above 70?'. The similarity was 0.869723975658, so the cached answer was returned even though the numeric predicate changed. The wording shares restaurant, score, and comparison language, which explains the high embedding similarity. Query signatures reduce this risk, but a richer structured predicate check would be safer.


```
[
  {
    "cached_query": "Which restaurant scored above 90?",
    "incoming_query": "Which restaurants scored above 70?",
    "similarity": 0.869723975658,
    "threshold": 0.75,
    "cached_answer": "verified answer for Which restaurant scored above 90?",
    "correct_answer": "miss by frozen label"
  }
]
```


Figure 8. Recorded qualifying false-hit example.

The 0.75 mean latency was 2537.92 ms, much higher than the other rows. The raw artifact does not include per-case timing samples or a separate cold-start marker, so the exact cause cannot be proven. The most plausible explanation is first-use Ollama embedding/model overhead, because the provider cache is cold at the beginning of the sweep. The reported value is end-to-end lookup time for each case, including embedding generation and Redis KNN, not pure Redis KNN latency.

4.3 Threshold recommendation

Under the required cost model, I would ship 0.95 as the measured lowest-cost threshold. I would keep the signature, restaurant-scope, and data-version checks enabled and would not interpret 0.95 as sufficient by itself: three false hits remain and ten expected hits are missed. A production policy could require a higher-confidence threshold plus structured predicate validation for score, date, and restaurant filters.

The separate fallback false_hit_example.json is retained as historical diagnostic evidence only. It is not used as the qualifying live Ollama false-hit example. The qualifying result above is the primary evidence.

5. Stale-answer experiment and version invalidation

The live MySQL/Redis sequence cached a restaurant answer, changed the real restaurant location, queried again without invalidation, and observed the old answer. The restaurant data version was then incremented. The next query missed the old cache entry and returned the updated location. The invalidation is scoped to one restaurant instead of deleting every cache record.

```
203:         return json.loads(raw)
204:
205:     def invalidate_restaurant(self, restaurant_id: int) -> str:
206:         key = f"{DATA_VERSION_KEY}:{restaurant_id}"
207:         return str(self.redis.incr(key))
208:
209:     def put(self, query: str, answer: str, signature: str, restaurant_id: int | None = None) ->
    str:
210:         cache_id = uuid.uuid4().hex
211:         record = {
212:             "query": query,
213:             "answer": answer,
214:             "signature": signature,
215:             "embedding": self.provider.embed(query),
216:             "data_version": self._current_version(restaurant_id),
217:             "restaurant_id": restaurant_id,
218:             "created_at": time.time(),
219:         }
220:         key = f"{CACHE_KEY}:{cache_id}"
221:         if hasattr(self.redis, "json"):
222:             self.redis.json().set(key, "$", record)
223:         else:
224:             self.redis.set(key, json.dumps(record))
225:         self.redis.sadd(f"{CACHE_KEY}:ids", cache_id)
226:         return cache_id
227:
228:     def _offline_lookup(
229:         self, signature: str, embedding: list[float], restaurant_id: int | None,
    current_version: str
230:     ) -> tuple[dict[str, Any] | None, float]:
231:         """Explicit in-memory fallback for unit tests without Redis Search."""
232:         best: tuple[dict[str, Any] | None, float] = (None, 0.0)
233:         for raw_id in self.redis.smembers(f"{CACHE_KEY}:ids"):
234:             cache_id = raw_id.decode() if isinstance(raw_id, bytes) else str(raw_id)
235:             record = self._get_record(f"{CACHE_KEY}:{cache_id}")
```

```
{
  "first": {
    "answer": "location=San Jose",
    "is_cached": false,
    "similarity": 0.0,
    "lookup_latency_ms": 40.689800000109244,
    "agent_latency_ms": 43.56290000032459,
    "total_latency_ms": 90.773399999307,
    "agent_executed": true,
    "cache_id": "a4e64720d247438cb3519b6f921b5125"
  },
  "stale_before_invalidation": {
    "answer": "location=San Jose",
    "is_cached": true,
    "similarity": 1.0,
    "lookup_latency_ms": 11.806600000454637,
    "agent_latency_ms": 0.0,
    "total_latency_ms": 11.808900000687572,
    "agent_executed": false,
    "cache_id": "a4e64720d247438cb3519b6f921b5125"
  },
  "stale_after_database_update": {
    "answer": "location=San Jose",
    "is_cached": true,
    "similarity": 1.0,
    "lookup_latency_ms": 30.561600000510225,
    "agent_latency_ms": 0.0,
    "total_latency_ms": 30.56410000044707,
    "agent_executed": false,
    "cache_id": "a4e64720d247438cb3519b6f921b5125"
  },
  "fresh_after_invalidation": {
    "answer": "location=HW9-Updated-City",
    "is_cached": false,
    "similarity": 0.0,
    "lookup_latency_ms": 14.149300000099174,
    "agent_latency_ms": 5.908300000555755,
    "total_latency_ms": 25.32840000003489,
    "agent_executed": true,
    "cache_id": "b8777d764a74480babc1897b69cc4de5"
  },
  "old_answer_reused_without_invalidation": true,
  "updated_answer_after_invalidation": true
}
```


Figure 9. Live stale-answer and data-version evidence.

Version markers are preferable to a global delete because unrelated restaurant answers can remain available. The trade-off is that writes must increment the correct version marker; a missed increment can leave stale data available.

6. Part 4 - Self-improving multi-agent system

The Answering Agent reads the reference dataset and optional lessons. The Evaluator Agent compares an answer with the reference fact. A REVISE result allows exactly one revised answer and a second evaluation. A successful repair can create a lesson, but lesson validation must prevent a local improvement from damaging previously correct or unrelated questions.

```
069: class SelfImprovingSystem:
070:     def __init__(self, reference_path: str | Path, lessons_path: str | Path, model_client: Any |
       None = None):
071:         self.answerer = AnsweringAgent(reference_path, model_client=model_client)
072:         self.evaluator = EvaluatorAgent(reference_path)
073:         self.lessons_path = Path(lessons_path)
074:         self.lessons = self._load_lessons()
075:
076:     def _load_lessons(self) -> list[dict[str, Any]]:
077:         if not self.lessons_path.exists():
078:             return []
079:         return json.loads(self.lessons_path.read_text(encoding="utf-8"))
080:
081:     def _save_lessons(self) -> None:
082:         self.lessons_path.parent.mkdir(parents=True, exist_ok=True)
083:         self.lessons_path.write_text(json.dumps(self.lessons, indent=2), encoding="utf-8")
084:
085:     def run(self, question: str, *, save_lesson: bool = True) -> dict[str, Any]:
086:         started = time.perf_counter()
087:         relevant = list(self.lessons)
088:         initial = self.answerer.answer(question, relevant)
089:         first_eval = self.evaluator.evaluate(question, initial)
090:         attempts = 1
091:         final = initial
092:         final_eval = first_eval
093:         if first_eval.result == "REVISE":
094:             final = self.answerer.answer(question, relevant, first_eval.feedback)
095:             final_eval = self.evaluator.evaluate(question, final)
096:             attempts = 2
097:         if save_lesson and first_eval.result == "REVISE" and final_eval.correct:
098:             self.lessons.append({
099:                 "lesson_id": f"lesson_{uuid.uuid4().hex[:8]}",
100:                 "failure_pattern": first_eval.explanation,
101:                 "correction": first_eval.feedback,
102:                 "created_at": time.strftime("%Y-%m-%d"),
103:                 "use_count": 0,
104:             })
105:             self._save_lessons()
106:         elif save_lesson and self.lessons:
107:             # Persist use_count changes when a previously validated lesson is reused.
108:             self._save_lessons()
109:         return {
110:             "question": question,
111:             "initial_answer": initial,
112:             "evaluation": first_eval.result,
```

Figure 10. SelfImprovingSystem repair loop excerpt.

6.1 Dataset separation and bounded repair

training_data.json, validation_data.json, and test_data.json contain questions only. Expected facts are kept in reference_data.json. The held-out test questions are used for comparison, not for lesson creation or prompt tuning. The controlled repair used the question 'Which restaurants scored below 70?'. The first answer was intentionally incorrect, the evaluator returned REVISE with a verified fact, the revised answer passed, and the run used two attempts.


```
{
  "question": "Which restaurants scored below 70?",
  "initial_answer": "incorrect answer",
  "evaluation": "REVISE",
  "feedback": "Use this verified fact: Sunrise Deli scored 65",
  "revised_answer": "Sunrise Deli scored 65",
  "final_evaluation": "PASS",
  "attempts": 2,
  "latency_ms": 16.251
}
```


Figure 11. Controlled repair result.

6.2 Lesson persistence, reuse, and validation

Lessons contain lesson_id, failure_pattern, correction, created_at, and use_count. The controlled run recorded one created lesson and use_count=1 after reuse. A deliberately rejected lesson was not retained. The current lessons.json may be reset by later experiments, so the raw agent_validation.json is the authoritative record for the reuse event.

```
        lesson_count_after_repair: 1
        lesson_use_count_after_reuse: 1
        rejected_lesson_file_exists: false
```

Figure 12. Lesson creation and rejection evidence.

6.3 Four-variant comparison

Variant	Correct	Incorrect	Avg attempts	Avg latency ms
Baseline	9	1	1.00	0.0076
Evaluator Repair	10	0	1.10	0.0103
Lesson Memory	10	0	1.10	2.4556
Validated Improvement	10	0	1.10	1.8157

Figure 13. Four variants evaluated on the same ten-question held-out set.

Baseline produced 9/10 correct. Evaluator Repair, Lesson Memory, and Validated Improvement produced 10/10. The average attempts are calculated from individual runs. The timing boundary uses time.perf_counter around the local answer/evaluation path; it includes deterministic reference operations and lesson-file work where applicable, but not Ollama network latency. The values are therefore local execution timings, not LLM service benchmarks. Validated Improvement was strongest on accuracy, but the sample is small and does not establish generalization.

7. Testing and regression audit

Command/check	Result	Status	Notes
python -m pytest tests/test_hw9.py -q	4 passed	PASS	View counter, cache safeguards, bounded age
ruff check scoped HW9 files	All checks passed	PASS	Includes KNN and report builders
python -m compileall ...	Exit 0	PASS	Relevant source and scripts
docker compose ... config --quiet	Exit 0	PASS	Compose syntax
live_knn_validation.py	status PASS	PASS	Redis KNN, stale, empty, concurrent
qualifying sweep	five rows redis_knn=true	PASS	Live Ollama and Redis
SQLite regression suite	40 passed, 2 failures	PARTIAL	Earlier failures retained and disclosed
Full default suite	collection blocked	RECORDED	Local MySQL authentication configuration

Figure 14. Test and verification matrix.

The focused HW9 suite is green. The broader earlier-homework suite was not fully green: 40 tests passed and two failed, and the default suite could not complete collection under the local MySQL authentication configuration. I do not relabel those outcomes as a complete repository pass.

8. Runtime evidence and required artifacts

The runtime figures show the live Swagger surface and health response, while the source panels show relevant excerpts from the final HW9 files. The raw JSON files provide the detailed view-counter, semantic-cache, KNN, stale-invalidation, agent, threshold, Redis-index, and Docker results.

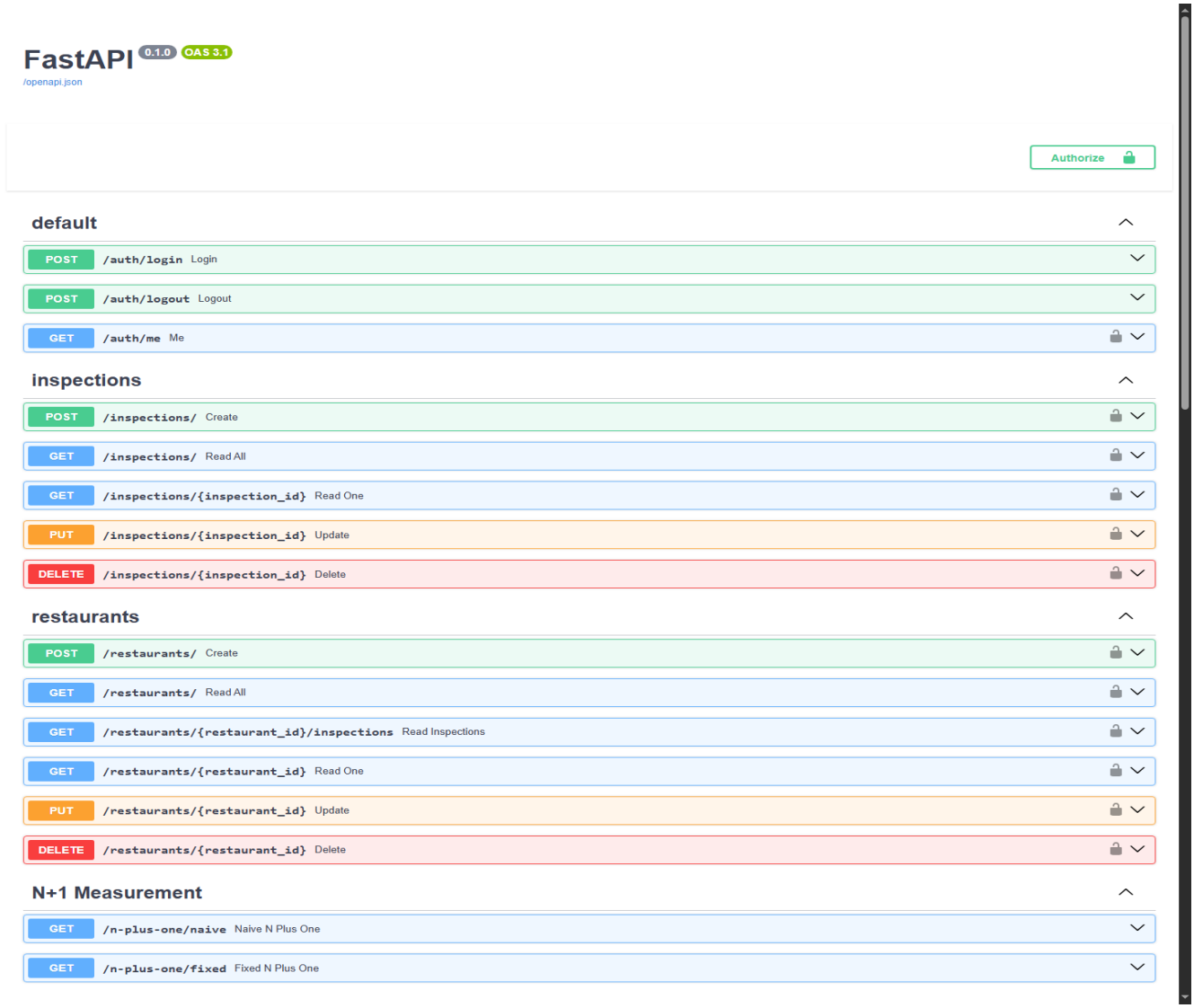


Figure 15. Live FastAPI Swagger surface.

8.1 Reproducibility

```
docker compose -f docker-compose.hw9.yml up -d --build
python -m pytest tests/test_hw9.py -q
ruff check hw9_cache.py hw9_agents.py routers/hw9.py tests/test_hw9.py
python -m compileall -q hw9_cache.py hw9_agents.py routers/hw9.py
python run_hw9_live_knn_validation.py
python run_hw9_qualifying_sweep.py
docker compose -f docker-compose.hw9.yml config --quiet
```

The qualifying sweep should not be rerun casually because live embedding calls are slow and the result is already preserved. The command is recorded for reproducibility, not as a request to change the committed dataset or labels.

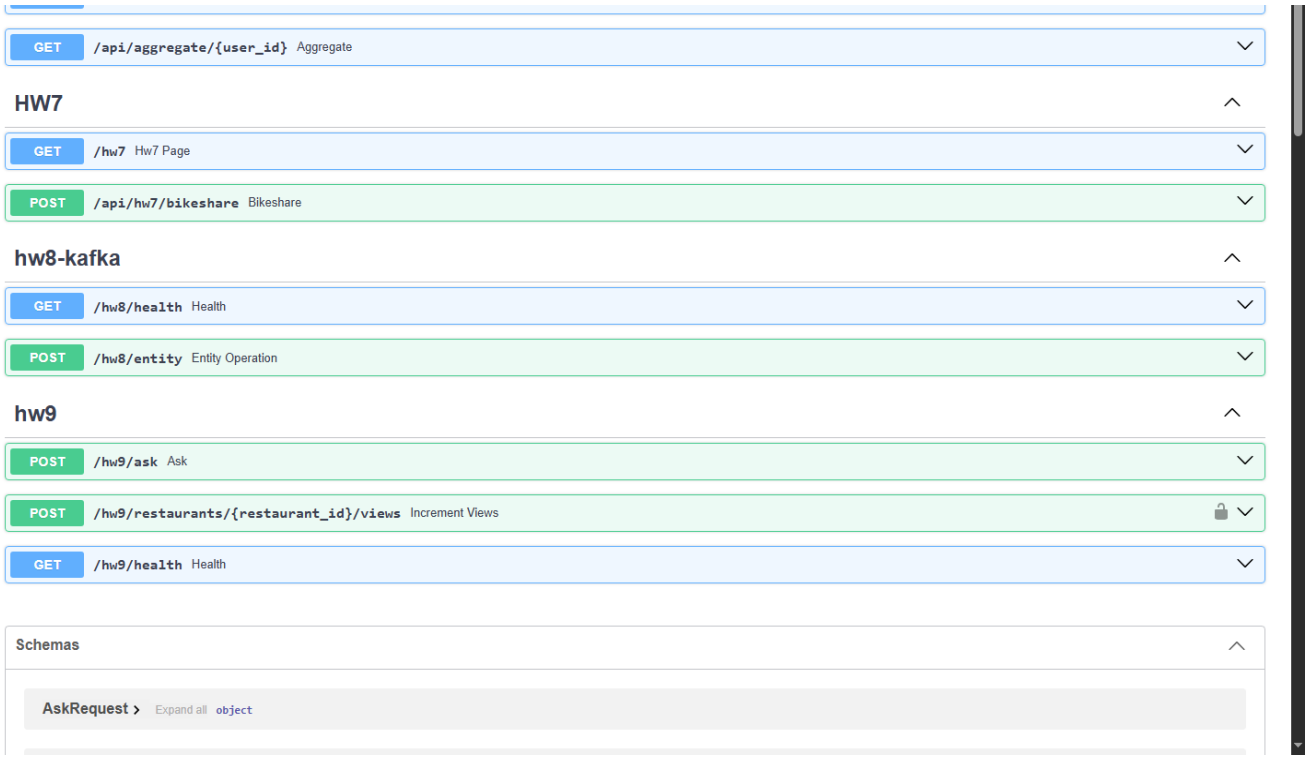


Figure 16. HW9 endpoint group in Swagger UI.

9. AI use disclosure and verification

AI use disclosure

1. AI assistance was used to inspect the assignment and existing repository, draft implementation changes, and help write tests and documentation.
2. The student remains responsible for the design, code review, execution, and submitted results.
3. The report separates live Redis, Ollama, and database results from deterministic local tests and identifies the commands and raw artifacts used for each result.
4. I reviewed the implementation, experiment outputs, and report contents before submission.

Dataset-freeze verification: commit a7533e7d5eb1df286b77450ce7999d0ad6a5a12d contains only cache_cases.yaml. The final package includes the HW9 implementation, experiments, report, and supporting evidence on the dedicated hw9 branch. Protected tags remain hw6=0307ac55b1ec9d6e48561a43e5671e21c09fd708, hw7=b78001eaa5c0f071eaf1788702c019d67983860a, and hw8=1b843e76d268dd9076230411e098e96ed43b9a8f.

10. Final rubric checklist and conclusion

Requirement	Status	Evidence
Redis INCR, prefix, tenth-write sync	PASS	Live view JSON, source, focused tests
Redis error and missing-restaurant handling	PASS	Source and unit tests
Concurrent view increments	PASS	Live counter JSON
Semantic cache at /hw9/ask	PASS	Live API JSON and route/source
Actual Redis HNSW KNN	PASS	live_knn_validation.json; qualifying rows
Signatures, scope, data versions	PASS	KNN source and stale evidence
Frozen dataset committed before sweep	PASS	Commit a7533e7
Five-threshold live sweep	PASS	threshold_sweep_qualifying_knn.json
False-hit analysis	PASS	Qualifying false-hit records
Stale-answer invalidation	PASS	live_invalidation_mysql_redis.json
Bounded two-attempt repair	PASS	agent_validation.json and source
Lesson reuse and validation	PASS	agent_validation.json
Four independent variants	PASS	self_improvement_variants.json
Source excerpts and runtime figures	PASS	Report panels and live evidence
Regression results	PASS	Focused HW9 and recorded broader-suite results

The HW9 implementation, qualifying live KNN sweep, and supporting experiments are included in this submission. The measured lowest-cost threshold is 0.95, with the accuracy trade-off shown in the results table. The four-variant timings measure deterministic local answer, evaluation, and lesson-file work, so they are reported as local execution measurements rather than model-service benchmarks.

Artifact inventory: source files at the repository root; datasets under data/hw9; machine-readable evidence under reports/hw09/raw; runtime images under reports/hw09/evidence; RUN_LOG.txt, METRICS.md, AI_USE.md, EVIDENCE_MANIFEST.md, and REQUIREMENTS_CHECKLIST.md under reports/hw09.